SAATHI CARE AI

An AI-assisted care coordination companion for older adults and their trusted people.

A detailed problem statement, original project proposal, technical blueprint, and submission plan for **Next Byte Hacks V5**.

> **THE PROJECT IN ONE SENTENCE**
>
> Help an older person ask for support easily, share only what they choose, and see who has taken responsibility for responding.

Based on a close reading of:
Suraksha_AI_Elder_Care_Detailed_Analysis.pdf, all 7 pages, plus current hackathon pages and primary technical sources.

Prepared: 18 October 2026
Status: Proposed project; no implementation or user testing is claimed.
Working name: Saathi Care AI; naming availability has not been checked.

INDEPENDENCE / CONSENT / ACCOUNTABLE RESPONSE

01 / RECOMMENDATION & READING GUIDE

What your project should be

Build Saathi Care AI: a voice-friendly web application that helps an older adult manage simple routines and request support, while a trusted care circle receives permission-checked alerts and explicitly accepts responsibility for responding. The central demonstration should follow one request all the way from the older person to an acknowledged caregiver action.

The proposed entry combines an older-adult interface, a caregiver interface, and a small backend that stores reminders, permissions, and incident state. AI translates everyday language into a structured request. Deterministic application code checks permission and manages the response. A visible timeline shows what happened, who was contacted, and whether someone actually accepted the request.

Your most useful additions are a **care-circle drill**, a **permission preview**, and an **acknowledgment-to-resolution handoff**. These make the project demonstrable even without a smartwatch. They also shift the engineering focus toward whether a care arrangement works when needed.

> **ORIGINALITY**
>
> Use Suraksha as an attributed case study. Create your own implementation during the eligible event period. Reusing its name, screenshots, completed application, or prize claims would not establish a new project.

Read this part	Pages
Source analysis, problem statement, and intended users	3–5
Proposed solution, event fit, and prototype scope	6–8
User journeys, AI boundaries, consent, and architecture	9–12
Workflow, data model, screens, and device feasibility	13–16
Testing, build plan, impact, and demo	17–20
Reusable project description and future roadmap	21–22
Sources and final submission checklist	23–24

Reading convention: “The supplied PDF describes” refers to the reference project. “Proposed,” “target,” and “should” describe your future implementation. All personas, events, timing examples, budgets, and evaluation targets are illustrative unless identified as externally verified.

02 / DEEP ANALYSIS OF YOUR PDF

What Suraksha teaches—and what it does not prove

The uploaded document is a seven-page technical explainer about an existing project, rather than a fresh competition problem statement. Its cover attributes Suraksha to Team StarBugs. The official First Commit gallery independently lists Suraksha as the Ship It first-prize project. [5]

PDF evidence	Interpretation for your project
Page 1: independence, assistance, privacy	Keep the older adult as the primary decision-maker. Family reassurance is a secondary benefit.
Page 2: distance, unnoticed problems, difficult interfaces	The core need is a usable route from uncertainty to appropriate human support.
Page 3: voice, device signals, reminders, consent, response	A useful care product must execute and track actions; conversation alone is incomplete.
Page 4: separate interaction, authorization, and workflow paths	Keep language interpretation separate from permission decisions and persistent process state.
Page 5: ordered contact and explicit acknowledgment	Sending an alert is only one step. Track whether a person actually takes responsibility.
Page 6: AWS stack and indexed due-work retrieval	Choose infrastructure for a working vertical slice; avoid adding services just to reproduce the diagram.
Page 7: validation gaps and proposed improvements	Treat hardware accuracy, delivery reliability, accessibility, and field readiness as open questions.

The main gap worth addressing

The PDF explains how a system can contact people, but leaves room to demonstrate whether the care circle is ready: Are contact preferences current? Can a person accept or decline? What happens after acceptance? Can the older adult inspect exactly what someone else will see? Your project should make these questions visible and testable.

The document does not supply a runnable specification, independently measured accuracy, validated health outcomes, or independently inspected source code. It reports that the original repository was unavailable during its research. This analysis did not audit that repository or operate the original application. The developer article supports the broad design context, not clinical or reliability claims. [4]

> **ANALYSIS OUTCOME**
>
> Caregiver drill mode already appears as a future idea on page 7. Credit that inspiration. Your contribution is the newly built workflow, permission interface, tests, and demonstrated behavior—not a claim that the idea has never existed.

03 / COMPLETE PROBLEM STATEMENT

The gap between noticing and helping

Older adults living independently may need occasional support with everyday routines or unexpected situations, while trusted relatives may be geographically distant, busy, or unsure who is responsible. The problem is the absence of a simple, privacy-respecting process that turns a request or concerning event into timely, acknowledged human follow-through.

In a common situation, an older adult has a phone, a few family contacts, and a routine that changes from day to day. A relative may call to check in, but a missed call has several possible explanations. The person may be resting, their phone may be charging, or they may need help. Repeated calls can become intrusive, while silence can produce uncertainty. Neither silence nor a missing sensor reading proves an emergency.

When a message is sent to several relatives, everyone may assume someone else will respond. One person may read the alert but be unable to help. Another may call without informing the group. A system can display a successful notification while the practical question—"Who is handling this?"—remains unanswered. This is a coordination failure as much as a sensing problem.

Five connected barriers

- **Interaction burden:** small controls, complicated navigation, and repeated data entry can make routine help harder to request.
- **Unclear ownership:** a broadcast message does not assign responsibility, record a decline, or confirm follow-through.
- **Loss of context:** fixed alarms cannot always express requests tied to a person's routine; fragmented messages lose relevant details.
- **Privacy tradeoffs:** a person may want a neighbour to help without exposing medication notes or private conversations.
- **Uncertain system state:** missing connectivity, stale events, and failed deliveries can look like normal inactivity unless explicitly shown.

WHO projects that one in six people globally will be aged 60 or older by 2030. This establishes the scale of ageing, but it does not prove demand for this particular application. User interviews and usability testing are still needed. [6]

> HACKATHON PROBLEM STATEMENT
>
> How might we help an older adult request and coordinate everyday support through a simple interface, while ensuring that only approved information reaches the right people and that every request has a visible response status?

Scope: daily support, check-ins, and coordination. The prototype does not diagnose illness, establish whether a fall occurred, verify medication ingestion, or provide a guaranteed emergency response.

04 / USERS & DESIGN ASSUMPTIONS

Who you are building for

Begin with one older adult who can express preferences, a primary caregiver, and a backup contact. This narrow household model is sufficient to test the complete interaction. The following people are fictional design personas, not interviewed participants.

Persona	Need and constraint	Product response
Meera, 70 Older adult	Lives independently; wants easy reminders and control over sharing. May prefer speaking to typing.	Large controls, optional voice, simple confirmation, visible sharing settings.
Arjun, 35 Primary caregiver	Lives elsewhere; cannot watch a dashboard continuously. Needs to know when action is required.	Actionable requests, accept/decline controls, context limited to permission.
Leela, 48 Trusted backup	Can sometimes help nearby but should not receive private care notes.	Availability setting and a minimal request to check in.

Jobs the product should help people complete

Older adult: “When I need help or want a reminder, let me express it simply and tell me what will happen next.” **Caregiver:** “When I receive a request, show what I can do, whether someone else is handling it, and how to update the situation.” **Backup contact:** “Let me help within my availability without giving me unnecessary personal information.”

Assumptions to validate

- The older adult is comfortable using a phone or tablet for one or two familiar actions; a family member can assist with initial setup without taking over consent.
- At least one trusted person agrees to participate. Merely adding a phone number does not establish willingness or availability.
- An internet connection is available during the main demonstration. Disconnections are visible and queued actions are clearly labelled.
- English text input is the baseline. English voice is the first speech target; Hindi/Hinglish is a stretch feature subject to actual device and language testing.

Small discovery exercise before building

Ask three to five willing adults or caregivers about their current check-in process, unwanted sharing, and what “I am responding” should mean. Use fictional scenarios and avoid collecting medical records. Observe whether people understand the proposed buttons. Record actual feedback separately from your assumptions. A small convenience sample guides interface decisions; it is not representative evidence of market demand.

> **USER-CENTRED SUCCESS**
>
> Initial success means a participant can request support, understand who will see it, and identify who accepted it. It does not mean the app has proven that a person is safe.

05 / PROPOSED SOLUTION & DIFFERENTIATION

Saathi Care AI: a care circle that follows through

Saathi Care AI is a proposed responsive web app with two role-specific experiences: a simple personal companion and a focused caregiver workspace. The backend converts a confirmed request into a reminder or care incident, applies current permissions, and tracks the human response until an explicit outcome is recorded.

The product has three connected loops

- **Routine loop:** ask for a reminder, confirm its meaning, receive it, and optionally mark it complete. Event-linked timing uses an explicit check-in in the MVP.
- **Support loop:** request help or trigger a labelled test event, contact an approved responder, wait for acceptance, and hand off when needed.
- **Trust loop:** preview what each contact may see, update permissions, and inspect a record of decisions and disclosures.

Inspired by the PDF	Your proposed emphasis	Evidence to show
Voice assistance and reminders	Confirmed, structured actions with an accessible text alternative	A spoken or typed request becomes a persisted reminder.
Family escalation	Acceptance, decline, follow-up deadline, and explicit resolution	An unanswered request moves to a backup and remains open after acceptance.
Consent checks	Per-contact preview and revocation during an active incident	A restricted contact sees a reduced payload; a revoked contact loses access.
Future caregiver drill idea	A complete, repeatable rehearsal with a labelled report	A test case records response steps and identifies an unanswered contact.

The proposed differentiation is a product hypothesis, not a claim of worldwide novelty. Many products address reminders, alerts, and caregiving. A broader competitor review would be needed before making a “first ever” claim. In this hackathon, originality should be visible through your own code, interaction design, tested workflow, and honest attribution.

> **SIGNATURE SCENARIO**
>
> The memorable demonstration: Meera allows Leela to receive help requests but blocks private notes. Arjun does not respond. Leela receives only the permitted request, accepts responsibility, and later records the outcome. Meera can inspect the entire sharing history.

Suggested attribution: “Inspired by the published Suraksha elder-care case study by Team StarBugs. Saathi Care AI is a separate implementation focused on permission previews, care-circle drills, and explicit caregiver handoffs.” Use that statement only if the resulting work matches it.

06 / HACKATHON FIT & VERIFIED DETAILS

How this fits Next Byte Hacks V5

The overview welcomes open-ended software or hardware projects, including AI and social-good work. It requests a project description, a 1–3 minute demo, a public GitHub repository with setup instructions, 2–3 screenshots, and a Devpost page. Its visible judging section lists Impact. [1]

The rules specify teams of 1–4, newly created work built during the event, accessible source, and a 2–5 minute video. They list four equally weighted criteria: Innovation & Creativity, Technical Complexity, Functionality & Execution, and Presentation & Clarity. Previously completed or submitted projects are excluded. [2]

The schedule lists submissions from 6 October 2026, 12:00 a.m. EDT, until 31 October 2026, 5:00 p.m. EDT. The deadline converts to **1 November 2026, 2:30 a.m. India Standard Time**. Use an internal target of **31 October, 6:00 p.m. IST** to leave an upload buffer. [3]

Page mismatch	Recommended handling
Overview eligibility panel says students and legal majority; descriptive copy and rules mention 13+ / regional requirements. [1,2]	Your eligibility cannot be confirmed from these inconsistent statements. Ask the organizer which conditions govern before relying on eligibility.
Video ranges differ. [1,2]	Record a 2 minute 45 second demo, which fits both posted ranges.
Overview and rules show different judging descriptions. [1,2]	Prepare evidence for the four detailed criteria and show a clear human benefit.

How to position this proposal

Impact: make responsibility visible after a help request. **Creative approach:** show the older adult a contact-specific sharing preview. **Engineering:** demonstrate persistent state, authorization, retries, and race handling. **Execution:** finish one reliable flow across two accounts. **Presentation:** show observable behavior and identify all simulated inputs.

The event fit is a reasoned assessment of the proposal, not organizer approval or a prediction of winning. Preserve the new project's commit history and note any earlier research or reused libraries. If you already completed an implementation before the allowed period, seek organizer clarification rather than treating a rename as a new entry.

> DATE CHECK
>
> As of 18 October 2026, the posted submission window is still open. Recheck the official pages before final submission because organizers can update details.

07 / THE PROTOTYPE CONTRACT

Exactly what to build first

Build one demonstrable system before adding more intelligence or hardware. A feature belongs in the core scope only if it directly supports the request → permission → response → outcome journey. The prototype should operate with synthetic household data and two signed-in test roles.

Priority	Feature	Definition of done
P0	Household and role setup	Older adult and two contacts are linked through invitations; unrelated accounts cannot access the household.
P0	Simple personal home	“I am okay,” “Request help,” reminders, and text input work with large controls.
P0	AI request interpretation	A language request produces validated fields and a confirmation screen before saving.
P0	Routine reminders	A fixed-time reminder and a check-in-relative reminder persist and trigger once.
P0	Permission preview	Contact permissions determine both displayed data and server responses.
P0	Care incident workflow	Primary contact can accept or decline; timeout reaches an authorized backup.
P0	Ownership and outcome	Acceptance records an owner; resolution is a separate action with a minimal outcome note.
P0	Care-circle drill	A labelled simulator runs the same workflow with test contacts and produces a timeline.
P1	Voice input and playback	One tested language supports a complete request; text remains available.
P1	External notification	One configured channel is exercised on consenting test accounts and delivery states are recorded.
P2	Wearable / extra languages	Added only after core behavior is stable; actual support is demonstrated.

What to deliberately defer

Defer custom fall-detection model training, continuous vital-sign analysis, hospital integration, automated ambulance dispatch, medication recommendations, autonomous diagnosis, broad multilingual claims, and a full marketplace of carers. Each adds a separate validation or operational burden that can obscure the core project.

> **BUILD ACCEPTANCE**
>
> Minimum credible submission: a working interface for each role, real server-side permissions, durable incident state, one real AI action, and a repeatable drill. If alerts only appear inside an open browser, say so clearly.

Do not describe a button that inserts a test event as a functioning fall detector. It proves downstream coordination only. Likewise, a user clicking “done” on a reminder is self-report; it does not prove a medication was taken.

08 / USER JOURNEYS

Three stories that define the build

Journey A — a reminder linked to a routine

Meera types or says, “Remind me to water the plants twenty minutes after my morning check-in.” The assistant proposes a reminder with trigger = morning check-in and offset = 20 minutes. The screen reads back the interpretation and asks for confirmation. After saving, Meera taps “I am okay” at 8:10 a.m.; the backend records that event and calculates a due time of 8:30 a.m. The reminder appears once and can be marked done or postponed.

Important branches: If no check-in occurs, the reminder remains waiting for its trigger. If Meera wants a fallback clock time, that preference must be explicitly configured. Repeated taps do not create duplicate deliveries. A late event should be explained before an overdue reminder is delivered. Morning check-in is a self-report event, not proof that a wearable detected waking.

Journey B — a support request reaches an owner

Meera presses “Request help” and confirms a simple message. The backend creates an incident and contacts Arjun. His screen offers “I will respond” and “I cannot respond.” If he declines or the configured wait expires, the workflow checks who is currently authorized and available, then contacts Leela. Leela accepts. The incident shows her as the responder while remaining open until she records an outcome.

Important branches: A late acceptance from Arjun must not create two owners. An expired invitation cannot acknowledge a new incident. If Leela accepts but does not follow up, the system records an overdue follow-up and applies the household’s configured backup process. If nobody accepts, it displays “No responder confirmed,” not “Help is on the way.”

Journey C — privacy changes during a request

Meera permits Leela to receive basic support requests but not private care notes. A preview shows the exact allowed view. During an incident, Meera revokes Leela’s access. The backend checks the latest policy before the next disclosure or fetch; future access is blocked and any pending notification job is re-evaluated. Information already delivered cannot be recalled, so the interface explains that limit.

> **REHEARSAL MODE**
>
> The same three journeys run in drill mode with fictional data, conspicuous “TEST” labels, shortened timers, and test recipients. A drill validates the software path and human response process, not real-world emergency readiness.

These journeys define the first backlog and the demo. Every extra feature should explain which step it improves.

09 / THE ROLE OF AI

Use AI for language; use code for authority

The AI component should make interaction easier. It interprets a person's words into a small set of supported intents and can produce concise explanations from authorized structured facts. It should not independently decide that somebody is safe, invent medical advice, approve a disclosure, or suppress a direct request for help.

AI task	Allowed output	Required guardrail
Interpret a reminder	Intent, task text, trigger, offset, optional clock time	Validate schema; ask about missing fields; confirm before saving.
Interpret a sharing preference	Proposed contact and permission changes	Resolve the exact identity; preview changes; require explicit confirmation.
Explain an incident	A short summary of permitted event facts	Apply access checks before assembling context; do not add a diagnosis.
Conversational assistance	Navigation help and supported product actions	Keep emergency and clinical claims out of generated responses.

Example structured request

```
{
  "intent": "create_reminder",
  "task": "water the plants",
  "trigger": "morning_check_in",
  "offset_minutes": 20,
  "recurrence": "ask_user",
  "requires_confirmation": true
}
```

The model proposes this object; the server checks it against an allowlisted schema. The server resolves the signed-in person and household, enforces valid limits, and creates a review screen. It does not trust model-supplied user IDs, roles, consent flags, or database queries. If "every morning" was not stated, the app asks whether the request is one-time or recurring.

Prompt and tool controls

Treat transcripts, imported text, and caregiver notes as untrusted input. They cannot redefine system rules or grant permissions. Expose a few narrow actions such as propose_reminder and explain_permitted_timeline. Require application confirmation and authorization for actual writes. Limit retries and time spent waiting for the model; the help button and deterministic workflow remain available during model failure.

For a real integration, select a currently available model and verify account access, region, cost, speech languages, and output-schema support during setup. Do not make the entire prototype depend on one model identifier mentioned in the reference project.

> **HONEST AI CLAIM**
>
> Claim "AI-assisted request interpretation" if that path works. If the demo uses canned model output, label that component simulated; do not present scripted text as live AI.

10 / CONSENT & INFORMATION SHARING

Make permissions understandable and enforceable

Consent is a product interaction and a server-side rule. A contact may be allowed to receive a minimal support request while being unable to read private notes. Separate the permission to contact someone from permission to disclose each data category. An older adult should be able to understand and change both.

Information / action	Arjun: primary	Leela: backup
Receive basic help request	Allowed	Allowed
Accept an active invitation	Allowed	Allowed when invited
Read private reminder text	Only if explicitly allowed	Denied by default
Read private care notes	Only if explicitly allowed	Denied
View precise location	Separate explicit permission	Denied by default
Edit the older adult's sharing rules	Denied unless a separately designed delegation exists	Denied

This is an example configuration, not a default entitlement for family members. At setup, show contact-specific choices in plain language. An invitation should tell the recipient what participation involves and let them decline. The initial prototype assumes the older adult can consent; legal-representative or guardianship workflows are outside its scope.

The authorization sequence

Authenticate → identify household and resource → load current permission → check action and fields → construct the smallest allowed response → record the decision. Apply the same sequence to API reads, queued notifications, AI context, exports, and acknowledgment actions. Hiding a field in the interface is insufficient if the API still returns it.

Use a single policy module with deny-by-default behavior. A Cedar-based policy layer is a possible implementation choice consistent with the supplied PDF; a smaller deterministic policy module is acceptable if the scope and tests are explicit. Store policy version and contact identity with each decision. Avoid sending broad health details in lock-screen notifications.

Revocation and fallback

When permission changes, invalidate future access and re-evaluate queued messages. Keep a minimal audit entry of the change. Do not silently override a denial because an event is labelled urgent. Any exceptional sharing rule would require deliberate prior design and appropriate review; the MVP should use only explicitly authorized contacts and fields.

> **THE TRUST FEATURE**
>
> Preview example: "Leela can see that you asked for a check-in and when you asked. She cannot see your private notes." The preview should be generated by the same permission logic used for real delivery.

11 / SYSTEM ARCHITECTURE

One shared backend, three clear responsibilities

PERSONAL EXPERIENCE	CAREGIVER EXPERIENCE	TEST INPUT
Voice / text Check-in · reminder · help	Invitation · accept / decline Follow-up · outcome	Labelled event simulator Duplicate / delay / offline cases
↓ Authenticated application API ↓		
LANGUAGE SERVICE	**POLICY SERVICE**	**WORKFLOW SERVICE**
Parse supported intent Return a proposal	Check actor + resource Filter permitted fields	Persist transitions Run timers and retries
↓ Durable records and delivery attempts ↓		
STATE	**SCHEDULING**	**DELIVERY**
Users · consent · reminders Incidents · invitations · audit	Due reminders Timeouts · follow-up checks	Caregiver web view Optional push / email channel

Recommended implementation path

For a team familiar with AWS, use a responsive React/Next.js interface, Cognito identity, API Gateway and Lambda for trusted actions, DynamoDB for persisted records, and Step Functions **Standard** for the incident process. Add Bedrock for the language component and one speech path only after text works. This is a proposed stack, not a requirement imposed by the hackathon.

AWS documents callback-wait integration patterns for Standard workflows; Express workflows do not support that pattern. An authenticated callback endpoint should map a short-lived invitation to the internal workflow token. Keep that token on the server and enforce incident state and contact identity before continuing. [7]

Simpler fallback if AWS setup becomes the bottleneck

Use the same frontend with a small backend, a transactional database, and a persistent worker that claims due jobs. Model every transition in the database and store deadlines explicitly. Do not use browser timers or an in-memory list as the only record of an incident. Choose one implementation path; maintaining two architectures would waste the available build time.

A one-minute reminder sweep is sufficient for this demo. Query due work through an index rather than scanning all households. EventBridge Scheduler documents 60-second invocation precision; do not base a 10-second demo timeout on that scheduler. Use workflow waits or a persistent worker for accelerated drill timers. [8]

> **ARCHITECTURAL INVARIANTS**
>
> No permission decision belongs solely in a prompt, and no incident should disappear when a browser tab closes.

12 / INCIDENT STATE MACHINE

A notification is not a completed response

Persist an incident state machine so the application can explain exactly what it knows. Keep provider delivery status, caregiver acknowledgment, and actual resolution separate. A successful HTTP request to a notification service only proves that a request was accepted at that layer.

State	Meaning	Next transition
CREATED	Confirmed request or labelled test event stored	Validate event and current contact policy.
CONTACTING	One authorized invitation being attempted	Record provider result and start response deadline.
AWAITING_RESPONSE	No contact has taken ownership yet	Accept → ASSIGNED; decline/timeout → next contact.
ASSIGNED	A named caregiver accepted responsibility	Wait for follow-up or a recorded outcome.
FOLLOW_UP_OVERDUE	Accepted but no outcome by the configured deadline	Recheck policy; prompt owner or invite an approved backup.
RESOLVED	An authorized person recorded an outcome	Close active timers; retain a minimal history.
NO_RESPONDER	Contact list exhausted without acceptance	Show unresolved status and the agreed manual next step.
CANCELLED	Request withdrawn through an authorized action	Stop future work; inform already-contacted responders as permitted.
DELIVERY_BLOCKED	No configured delivery path succeeded	Surface failure; retry within bounds or use approved fallback.

Race conditions and retries

Give each source event a unique ID and every notification attempt a deduplication key. Create at most one active incident for a repeated event. Acknowledge by an atomic conditional update: the first valid acceptance wins, and later attempts receive the current owner state. An expired or revoked invitation cannot claim ownership. If a timeout and an acceptance arrive together, resolve them against the same persisted version.

An acceptance can cancel future invitations, but it cannot retract a message already received. Tell a second responder that someone else has accepted. On restart, recover pending deadlines from storage. Retry transient failures with a bounded policy; after repeated failure, expose a visible failure state instead of endlessly claiming that contact is in progress.

Demo timing versus real timing

Use a 10-second primary-contact wait and a short follow-up timer only in drill mode. Display “Accelerated test timing.” These values exist to make the demo readable; they are not recommended emergency response thresholds. Real deployment would require context-specific design with users and qualified care partners.

> **STATUS LANGUAGE**
>
> “Leela accepted this request” is a supported fact. “Meera is safe” is not implied by that acceptance.

13 / DATA & API BLUEPRINT

Store facts that explain the workflow

Use small structured records with clear ownership and timestamps. A database design should make permissions, pending work, and incident history easy to retrieve. A large unstructured chat log is a poor substitute for these records.

Record	Essential fields
User / household	id, role, household_id, timezone, language, invitation_status
Contact link	elder_id, contact_id, order, available, accepted_participation
Consent rule	subject_id, contact_id, action, allowed_fields, version, updated_at
Reminder	id, owner_id, text, trigger_type, offset, due_at, recurrence, status
Event	id, subject_id, type, source, occurred_at, received_at, is_test
Incident	id, event_id, subject_id, state, owner_contact_id, version, next_due_at
Invitation / attempt	id, incident_id, contact_id, expires_at, delivery_status, accepted_at
Audit entry	actor_id, action, resource_id, policy_version, decision, timestamp

API action	Authorization and behavior
POST /requests/interpret	Signed-in adult; return a proposal, not an executed action.
POST /reminders	Owner confirms validated fields; enforce idempotency.
POST /events	Authorized app/device; validate source, timestamp, schema, and test flag.
POST /incidents	Confirmed help request; create persistent workflow exactly once.
POST /invitations/{id}/accept	Invited contact only; verify expiry, policy, and current state atomically.
PATCH /consent/{contactId}	Authorized subject; version change and re-evaluate future disclosure.
GET /incidents/{id}	Authorize every read and return only permitted fields.

Store UTC timestamps and the person’s named timezone; show local times in the interface. Keep occurred_at separate from received_at so delayed events are not mistaken for fresh activity. A due-time index supports reminders and incident deadlines. Use conditional writes or transactions for ownership changes.

Separate test records from any future real-user environment. Do not expose service credentials, workflow tokens, or raw audio in browser bundles or logs. Use a proposed short retention period for demo records, such as deletion after the event, and make the choice visible. A real service would need a carefully reviewed retention and deletion policy.

> **TRACEABILITY**
>
> The audit trail should answer: who requested the action, which permission was checked, what information was released, and what happened next.

14 / SCREEN-BY-SCREEN EXPERIENCE

Design for calm, legible action

Aim for a small number of screens with predictable language. The app should feel understandable before the user has learned any AI terminology. These are proposed screen requirements, not finished mockups.

Screen	Main content	Primary action
Personal home	Greeting, next reminder, latest self-reported check-in, connection status	I am okay / Request help
Request review	Interpreted request, timing, recipients, fields to be shared	Confirm / Edit / Cancel
My care circle	Contacts, participation, order, permissions, availability	Preview sharing / Change permission
Caregiver inbox	Requests requiring action and current owner where permitted	I will respond / I cannot respond
Incident detail	Minimal context, contact timeline, ownership, outcome status	Record follow-up / Resolve
Drill console	Test mode badge, scenario selector, controlled clock, test contacts	Start drill / Replay
Sharing history	Plain-language allow/deny decisions and disclosures	Inspect details

Accessibility choices to implement

- Use large touch controls, readable type, strong contrast, clear focus states, and text labels alongside icons.
- Support keyboard navigation and screen-reader labels for primary controls. Announce significant status changes without repeatedly interrupting the user.
- Offer a text path alongside voice. Show a transcript for correction and use push-to-talk with a visible recording indicator.
- Do not rely on colour alone for urgent, pending, or resolved states. Use explicit wording and a timestamp.
- Keep destructive and sharing changes reversible where possible, with plain-language confirmation and visible feedback.

Use precise status language

Prefer “Last check-in: 8:10 a.m.” to a green badge saying “Safe.” Prefer “Waiting for Arjun to respond” to “Family informed.” Prefer “Device connection unavailable” to “No activity detected.” These phrases reduce the chance that a technical state is mistaken for a health conclusion.

The permission preview is a useful centrepiece: choosing Leela immediately shows the request as she would receive it. Use the same server-side field filter for preview and delivery, then test both. It should be possible to demonstrate privacy without opening a developer console.

> **USABILITY TARGET**
>
> Design target: a first-time participant can complete a check-in or accept a request with minimal assistance. Measure the actual result; do not claim accessibility compliance from visual appearance alone.

15 / HARDWARE FEASIBILITY & FAILURE MODES

Do not let a smartwatch block the project

The supplied PDF describes Galaxy Watch signals. That does not establish that your chosen watch, account, operating system, and SDK expose the same events. For the MVP, use a manual morning check-in, a confirmed help button, and a labelled event simulator. Add hardware only after demonstrating the downstream system.

Samsung's Sensor SDK overview lists raw and processed measurements and limits support to Galaxy Watch4 and later supported devices; it does not support an emulator. Its FAQ says capabilities can vary by model/software and that distribution requires the partnership process. Development mode is for testing. [9] [10]

The Health Data SDK is a separate route to selected data stored in Samsung Health, including sleep records. Stored sleep data does not by itself establish an immediate wake-event feed. Neither reviewed overview establishes that your app can subscribe to a ready-made real-time fall event. Verify a specific supported API and permission path on actual hardware before committing to that claim. [11]

Failure	Required prototype behavior
Device / browser disconnects	Show last contact time and an explicit offline indicator; do not infer wellbeing.
Same event delivered twice	Deduplicate; display one incident and one effective reminder.
Old event arrives late	Record both times; label it delayed and apply a defined freshness rule.
Speech transcript is incorrect	Allow correction and require confirmation before meaningful side effects.
Model unavailable	Keep manual check-in, reminders list, and help request available.
Notification rejected	Record failure, retry within bounds, and expose the failed delivery state.
Caregiver unavailable	Move to the next currently authorized contact; do not assume a read receipt means acceptance.
No eligible contacts remain	Show unresolved status and the household's pre-agreed manual next step.

A queued request on an offline phone must say "Not sent yet." After reconnection, deduplicate and identify any delay. A browser-based prototype should not promise continuous monitoring when the device is asleep, the application is closed, or permissions have been denied.

> **DEVICE CLAIM BOUNDARY**
>
> A synthetic fall-like event demonstrates event ingestion and response coordination. It provides no evidence of fall-detection sensitivity, specificity, or real-world reliability.

16 / VALIDATION PLAN

Test the care process, not just the happy path

Use synthetic data and consenting test accounts. The following are proposed acceptance checks; none has been executed for this report. A useful test verifies a user-visible outcome or a security boundary, rather than simply duplicating an implementation function.

Test	Pass condition
Confirmed reminder	Typed request creates correct persisted fields; cancellation creates nothing.
Event-linked timing	One check-in computes the intended due time; duplicate input does not double delivery.
Primary decline / timeout	Both paths invite the next allowed contact and preserve the history.
Two acceptances	Exactly one owner is assigned; the second person sees the current state.
Revocation during an incident	Future data fetches and queued disclosures obey the updated policy.
Restricted contact	Private notes never appear in API output, AI context, preview, or notification payload.
Cross-household access	Changing a resource identifier cannot retrieve another household's data.
Expired invitation	An old link or token cannot accept a different or expired request.
Worker restart	Pending incident and reminder jobs resume from persisted state.
Provider / model failure	Visible failure or fallback behavior; no false success message.
Resolution semantics	Acceptance does not close the incident; an explicit outcome does.
Drill isolation	Test events stay labelled and only reach the configured test recipients.

Small, honest evaluation dataset

Create 30 request examples: straightforward reminders, ambiguous timing, unsupported requests, permission changes, and attempts to bypass authorization. Report intent-and-field correctness, clarification rate, and blocked-action behavior separately. If voice is included, record transcription and interpretation failures separately. Thirty examples are a development check, not a broad language-quality benchmark.

Measures worth presenting

Record the number of passed workflow scenarios, duplicate incidents created, unauthorized disclosures in tests, and completion time for key tasks. Measure event ingestion to visible caregiver update on the test setup; report the number of runs and observed range or median. Separate provider delivery timing from human response time. Do not translate these results into claims of lives saved or clinical effectiveness.

> **GO / NO-GO CHECK**
>
> Suggested internal goals: all critical workflow and authorization cases pass; no duplicate incident in replay tests; the primary demo completes reliably in three consecutive runs. These are acceptance goals, not results.

17 / BUILD PLAN & OWNERSHIP

A focused plan from 18–31 October

This plan assumes you begin on 18 October 2026 and use the remaining window for an original build. Dates below are India Standard Time. Adjust daily effort to team capacity, but keep the dependency order: state and permission behavior first, then convenience features and presentation.

Dates	Work package	Concrete output
18–19 Oct	Finalize user story, new repo, data contracts, synthetic fixtures, interface sketch	One agreed MVP and documented attribution.
20–21 Oct	Authentication, household links, personal home, caregiver view	Two separate accounts access the correct household.
22–23 Oct	Reminder persistence, event handling, incident states, accept/decline	One complete request-to-response flow without AI.
24–25 Oct	Permission preview, revocation, deduplication, drill mode	A restricted backup contact and repeatable failure scenario.
26–27 Oct	Live AI interpretation; optional voice and one notification channel	Confirmed natural-language request works end to end.
28–29 Oct	Authorization and failure checks, usability observation, fixes	Recorded test results and clear known limitations.
30 Oct	Record demo, capture screenshots, finish README and submission draft	Reviewable submission bundle.
31 Oct	Fresh setup check, link verification, final upload before internal cutoff	Submitted entry with buffer for upload problems.

Suggested responsibilities

With three people, assign one to the personal/caregiver experience, one to state and permissions, and one to AI integration, testing, and demo assets. With two, combine interface and AI while the other owns backend and reliability. A solo builder should cut wearable work, advanced voice features, and multiple notification providers before reducing authorization or workflow completeness.

Fallback if less than a weekend remains

Build text input, a single household, permission filtering, durable help requests, two caregiver test accounts, and a labelled drill. Add one live model call to interpret reminders if access is ready. Use an in-app inbox with honest limitations. A coherent four-step workflow is stronger evidence than many unfinished screens.

> **SCOPE CONTROL**
>
> Freeze new features on 28 October. Spend the final days fixing the main flow, checking access boundaries, and making the explanation precise.

Keep dated commits, a short decision log, source credits, and a clear distinction between pre-existing libraries and your newly written application. Do not claim all code is original if templates or generated scaffolding were used.

18 / IMPACT, COST & SUSTAINABILITY

What value you can reasonably demonstrate

The initial value proposition is reduced coordination uncertainty: the older adult understands how to ask for support; contacts know who accepted; and sharing is limited to current permission. These are observable product outcomes. Whether the app improves wellbeing, reduces caregiver stress, or changes safety outcomes remains a future research question.

Stakeholder	Expected benefit	How to investigate
Older adult	Easier requests and more control over disclosure	Observe task completion and ask whether the sharing preview is understood.
Primary caregiver	Clear ownership and fewer ambiguous handoffs	Compare a fictional group-message scenario with the incident workflow.
Backup contact	Limited context and explicit participation	Check whether the person understands what is requested and can decline.
Family care circle	Ability to rehearse and inspect gaps	Run a drill and review unanswered invitations and follow-up states.

Cost planning without invented prices

Cost depends on model tokens, speech duration, notification traffic, database operations, workflow transitions, hosting, and logs. Use existing phones or laptops for the core prototype and defer a watch purchase. Check current provider pricing and account quotas before enabling paid services; this report does not quote a verified deployment price.

Planning formula: total operating cost = hosting + storage/requests + workflow execution + AI input/output usage + speech minutes + delivered messages + observability. For a hypothetical 10-household trial with four AI requests per household daily, that is 1,200 requests over 30 days. This is a workload example, not measured demand or a cost estimate.

Choose your own spending ceiling and configure alerts, model-call limits, restricted test recipients, and log retention. Avoid unbounded retry loops. Prefer prerecorded demo narration over repeatedly generating speech during rehearsal when it adds no product evidence.

Possible path after the event

Start with a small supervised usability pilot. If families find the workflow useful, explore a household subscription or partnerships with community support organizations. Treat willingness to pay, support staffing, onboarding effort, and liability as open business questions. Avoid implying that a recurring software fee buys an emergency service.

> **IMPACT CLAIM**
>
> Hackathon evidence should be a working process plus transparent test observations. Population statistics and plausible benefits establish relevance; they do not establish product-market fit.

19 / DEMO & PRESENTATION

A 2 minute 45 second story

Use two browser sessions with different roles and only synthetic household data. Keep the test label visible throughout. The audience should understand the person's need, see a complete handoff, and notice the permission boundary without needing to inspect source code.

Time	Show	Explain
0:00–0:20	Personal home and the three-person care circle	A request can go unanswered when responsibility is unclear.
0:20–0:45	Natural-language reminder → interpretation → confirm	AI helps express the request; the user confirms the action.
0:45–1:05	Leela's permission preview	Support requests are allowed, while private notes are withheld.
1:05–1:40	Start drill → primary timeout → backup invitation	The workflow persists, checks permission, and moves to another contact.
1:40–2:05	Leela accepts, then records follow-up and outcome	Acceptance and resolution are different; ownership is visible.
2:05–2:25	Revoke permission and attempt a restricted view	A server-side rule blocks access and the audit history records the decision.
2:25–2:45	Architecture and test summary slide	State what is implemented, simulated, tested, and still planned.

Suggested opening

"When an older parent asks for help, sending a notification is only the beginning. Families also need to know who is responding, and the parent should decide what each person can see. Saathi Care AI turns that request into a clear, permission-checked handoff."

Screenshots that prove behavior

Capture the personal home with a confirmed reminder; the sharing preview for a restricted contact; and the incident timeline showing the primary wait, backup acceptance, and recorded outcome. Use readable labels and timestamps. Do not use screenshots from the original Suraksha application as your own product assets.

Likely judge questions

Why AI? To interpret everyday requests; reliable workflow behavior remains in code. **What is new in your work?** Show your implemented preview, drill, and handoff behavior with attribution. **What if nobody responds?** Demonstrate an unresolved state. **What if AI is wrong?** Show confirmation and manual actions. **Do you detect falls?** Answer according to the actual device integration, and identify synthetic events.

> **DEMO RESILIENCE**
>
> Keep a backup screen recording and a seeded reset procedure. If a service fails during the presentation, identify the failure and use the labelled recording; do not imply recorded behavior is a live run.

20 / REUSABLE PROJECT DESCRIPTION

Submission draft: adapt after the build

Project title: Saathi Care AI — Consent-Aware Elder-Care Coordination
Tagline: Simple requests. Clear permissions. A caregiver who confirms.

Problem we are addressing

Older adults living independently may need help with daily routines or unexpected situations, while their trusted family members live elsewhere or have limited availability. Phone calls, reminders, and group messages do not always provide a clear record of who is responding. At the same time, sharing every personal detail with every contact can undermine the older person's privacy and independence. We want to make support easier to request, easier to coordinate, and easier to control.

What we propose to build

Saathi Care AI is an AI-assisted companion and care-circle coordination app. An older adult can create a reminder in everyday language, complete a simple check-in, or request help through a clear interface. The assistant turns supported language into a structured proposal that the user reviews before it is saved. Trusted contacts use a separate workspace to accept or decline support requests and record follow-up.

When a request is confirmed, the application checks which contacts are authorized and what information they may receive. It then invites the primary contact and waits for an explicit response. If that person cannot respond or the configured wait expires, the system can invite an approved backup. Acceptance assigns a named responder; resolution requires a separate outcome. A visible timeline explains every step.

What makes this implementation useful

Our proposed emphasis is understandable privacy and accountable follow-through. A contact-specific preview shows what information will be shared. A care-circle drill lets participants rehearse the workflow with synthetic events and shortened timers. Server-side permission checks, persistent incident state, and duplicate-event handling make the demonstration more than a conversational interface.

Inspiration and boundaries

This concept is inspired by the published Suraksha elder-care case study by Team StarBugs. The intended submission is a separate implementation developed for this event. It focuses on care coordination and user control. Simulated wearable inputs will be labelled, and the prototype will not claim medical diagnosis, validated fall detection, or guaranteed emergency response.

> **DRAFT STATUS**
>
> Before publishing, change future-tense statements only for features you have actually completed. Add your real stack, actual challenges, observed tests, and learning. Remove any capability that is still a plan.

Suggested "what we learned" prompts: Which ambiguous request required clarification? Which permission case failed initially? How did the state machine handle late replies? What did a participant misunderstand? Answer from your actual build experience.

21 / ROADMAP & REAL-WORLD BOUNDARIES

What comes after a credible prototype

The hackathon should establish that the interaction and coordination mechanism works on a controlled setup. It cannot establish readiness for unsupervised elder care. Future work should reduce uncertainty in stages rather than adding more features before the basic process is trusted.

Stage	Next work	Evidence needed
After the event	Supervised usability sessions, clearer consent, language improvements	Participants understand actions and permissions; observed issues are documented.
Integration pilot	Real notification channels, actual supported wearable events, recovery behavior	Measured delivery outcomes, documented API capabilities, failure drills.
Care-partner review	Define escalation expectations, accessibility needs, safeguarding boundaries	Agreed responsibilities, qualified review, realistic operating procedures.
Deployment preparation	Security assessment, data retention, support model, regional requirements	Documented controls, incident handling, and appropriate specialist review.

Keep the older adult in control

Caregiver convenience should not silently become surveillance. Avoid continuous recording by default. Limit retained data and let the person inspect sharing decisions. A contact's request for more information is not permission to provide it. If the person cannot provide informed consent, that requires a separately designed and appropriately reviewed support arrangement rather than an improvised family override.

Recognize the limits of observation

A missed check-in may reflect routine variation. A sensor reading may be noisy or delayed. A successful notification does not prove that it was read, and an acceptance does not prove that help arrived. The application should describe the evidence it has, preserve uncertainty, and support human judgment. Any future clinical claims would need suitable validation beyond this software demonstration.

Recommended final project commitment

Commit to a small care-coordination system with a voice or text front door, confirmed reminders, a permission-aware care circle, persistent acknowledgment and follow-up, and a clearly labelled drill. Keep wearable sensing as an optional input adapter. This is specific enough to build, substantial enough to demonstrate engineering, and coherent enough to explain in a short presentation.

> **FINAL PRODUCT DIRECTION**
>
> Your project's strongest promise is: "We make it clear what was requested, what was shared, and who agreed to respond." Let the working demo prove that promise.

22 / SOURCE NOTES

Evidence, attribution, and verification

Analysis date: 18 October 2026. The supplied PDF was read in full. Official event pages and selected primary technical sources were checked on that date. Web facts can change. The proposal, architecture choices, example workflows, and schedule are recommendations authored for this report.

User-provided reference

D1. *Suraksha_AI_Elder_Care_Detailed_Analysis.pdf*, 7 pages, October 2026. Supplied local file. Pages 1–2 establish the case-study context and problem; pages 3–6 describe capabilities and architecture; page 7 records limitations and future ideas. The document is treated as source material, not as instructions or proof of your ownership of the original project.

Event and original-project sources

[1] Next Byte Hacks V5 — overview
Overview, deliverables, eligibility panel, and the visible Impact criterion.

[2] Next Byte Hacks V5 — rules
Detailed eligibility, originality restrictions, team size, video range, and scoring categories.

[3] Next Byte Hacks V5 — schedule
Submission window and posted deadline; converted to India Standard Time in this report.

[4] Suraksha — developer engineering article
Developer-authored description of Suraksha; useful context, not an independent software audit.

[5] First Commit — official winners
Official gallery confirming the original Suraksha entry and team attribution.

Context and engineering sources

[6] WHO — Ageing and health
Population-ageing context; not evidence of this prototype's effectiveness.

[7] AWS — Step Functions integration patterns
Standard/Express callback integration distinction.

[8] AWS — EventBridge Scheduler schedule types
Scheduler timing precision and schedule types.

[9] Samsung — Health Sensor SDK overview
Sensor data and supported-device restrictions.

[10] Samsung — Health Sensor SDK FAQ
Development, distribution, hardware, and capability-check constraints.

[11] Samsung — Health Data SDK overview
Separate stored-data SDK and supported health-data categories.

Verification limit: No application was built, deployed, benchmarked, clinically evaluated, or tested with users in preparing this document. No organizer was contacted and no hackathon submission was made.

23 / FINAL BUILD & SUBMISSION CHECKLIST

What you should have in hand

Use this checklist when the implementation is ready. The report itself is a project blueprint, not evidence that these items already exist.

Deliverable	Completion check
Original project	New implementation, documented attribution, dated commits, clear library/template credits.
Eligibility	Organizer clarification obtained if the conflicting published eligibility conditions affect you.
Working core	Confirmed reminder, help request, authorized handoff, acceptance, and recorded outcome.
Permission evidence	Restricted contact preview, server-side filtering, and active revocation demonstrated.
Reliability evidence	Duplicate event, late acceptance, restart, and no-responder cases exercised.
AI evidence	One live interpreted request with validation and confirmation, or honest simulation labelling.
Hardware evidence	Actual connected device behavior shown, or synthetic event origin stated clearly.
Repository	Readable setup, environment-variable names without secrets, fixtures, architecture, licenses, limitations.
Video	Clear narration, two role views, visible test labels, and no unsupported performance claims.
Screenshots	Your own personal view, privacy preview, and incident timeline.
Submission text	Actual built features, tested stack, real challenges, learning, attribution, and future work.
Final link check	Repository and demo links open for reviewers; setup instructions work from a fresh environment.

A practical README structure

Project purpose → quick start → configuration → test accounts and synthetic fixtures → architecture → supported workflows → test results → known limitations → attribution and licenses. Include a reset command or documented reset procedure for drill data. Never put provider keys or real household information in the public repository.

The decision in one paragraph

Use the elder-care problem from your reference, but submit a distinct, newly built project: **Saathi Care AI**. Focus on a simple request interface, permission-aware sharing, explicit caregiver responsibility, and repeatable drills. Make one complete household scenario work convincingly before expanding to more languages, devices, or care services.

> **YOUR HACKATHON NORTH STAR**
>
> Build a system whose actions can be explained and demonstrated: one request, the right information, an authorized person, and a recorded outcome.